



COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

SUBMITTED BY

NAME : RIFAQ AJMAL
REG NO : 23MDBCS 435
SECTION : CS-A
SEMESTER : 6th

Date: 28/2/2026

LAB 5

Task 1

What is the role of MPI_Barrier in collective communication?

Answer:

MPI_Barrier is a synchronization function that forces all processes to stop at a certain point until every process reaches that point. Only after all processes arrive at the barrier, they continue execution. This ensures coordinated execution and prevents some processes from running ahead of others.

Task 2

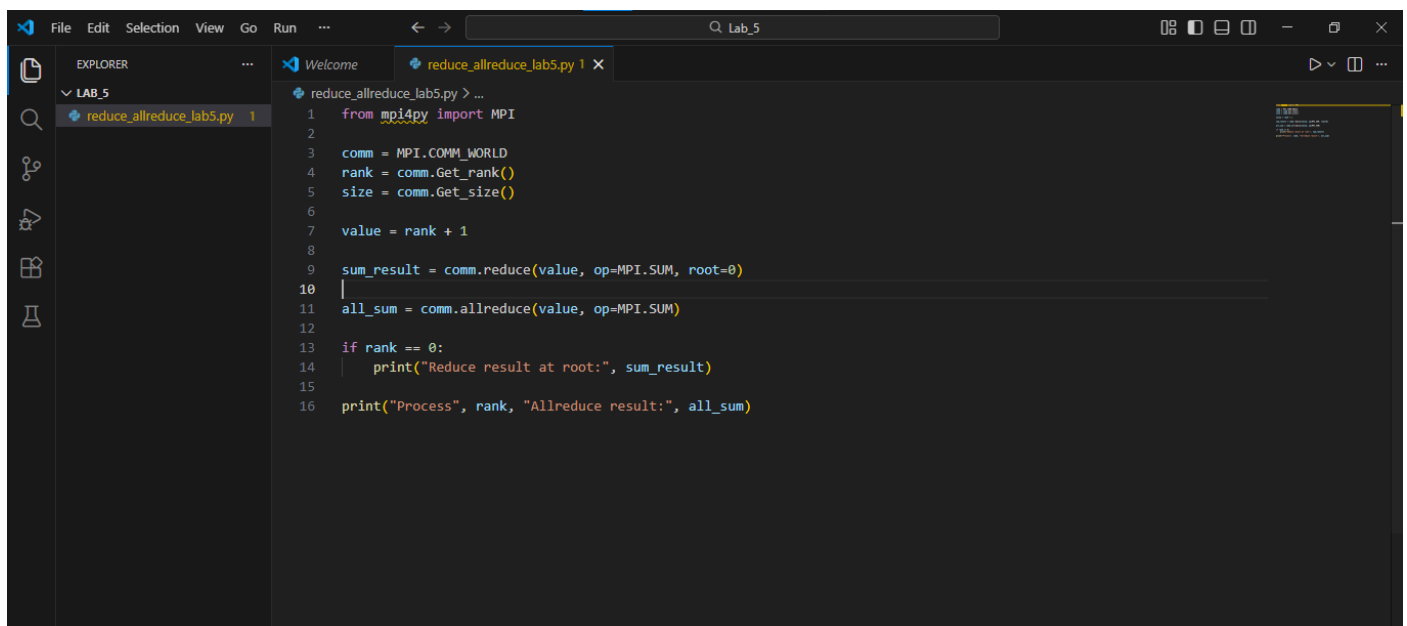
Why is synchronization important in parallel programming?

Answer:

Synchronization is important in parallel programming because multiple processes run at the same time. Without synchronization, processes may access shared data at different times, which can cause incorrect results, race conditions, or inconsistent data. Synchronization ensures processes execute in an organized and coordinated manner.

Task 3

Program for MPI_Reduce and MPI_Allreduce

A screenshot of a code editor window with a dark theme. The Explorer pane on the left shows a folder named 'LAB_5' containing a file 'reduce_allreduce_lab5.py'. The main editor area shows the code for 'reduce_allreduce_lab5.py'. The code imports MPI from mpi4py, initializes an MPI communicator, gets the rank and size, calculates a value based on the rank, performs a reduce operation, and then an allreduce operation. It prints the results for the root process and all processes.

```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5 size = comm.Get_size()
6
7 value = rank + 1
8
9 sum_result = comm.reduce(value, op=MPI.SUM, root=0)
10
11 all_sum = comm.allreduce(value, op=MPI.SUM)
12
13 if rank == 0:
14     print("Reduce result at root:", sum_result)
15
16 print("Process", rank, "Allreduce result:", all_sum)
```

Output

```
Anaconda Prompt

(base) C:\Users\Rifaq Ajmal>conda activate mpi_env
(mpi_env) C:\Users\Rifaq Ajmal>cd Desktop
(mpi_env) C:\Users\Rifaq Ajmal\Desktop>cd Lab_5
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_5>mpiexec -n 4 python reduce_allreduce_lab5.py
Reduce result at root: 10
Process 0 Allreduce result: 10
Process 3 Allreduce result: 10
Process 2 Allreduce result: 10
Process 1 Allreduce result: 10

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_5>
```

Difference between MPI_Reduce and MPI_Allreduce

- MPI_Reduce sends the final result only to the root process.
- MPI_Allreduce sends the final result to all processes.

So every process receives the result in MPI_Allreduce, while only the root process receives it in MPI_Reduce.

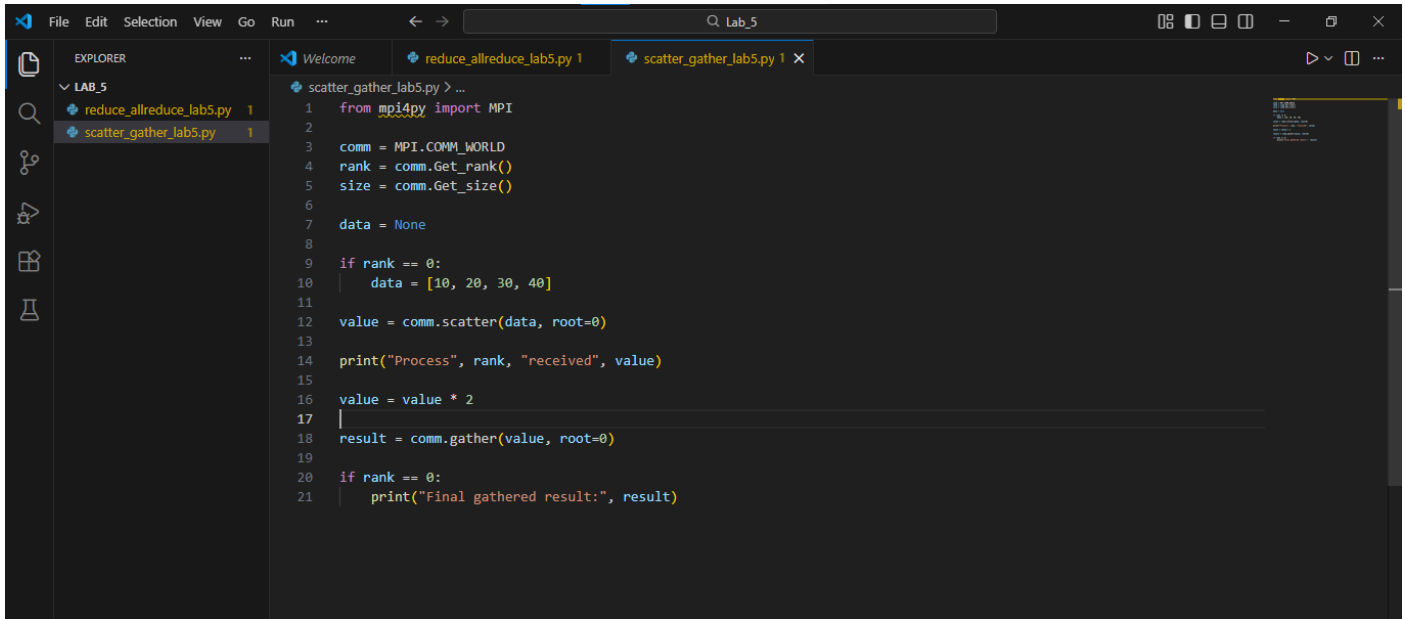
Task 4

What happens if MPI_Barrier is removed?

Answer:

If MPI_Barrier is removed, processes will not wait for each other. Some processes may execute faster and print output earlier, which can cause unordered output or incorrect synchronization. The program may still run, but execution will not be coordinated.

Task 5



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a folder named 'LAB_5' containing two files: 'reduce_allreduce_lab5.py' and 'scatter_gather_lab5.py'. The code editor shows the contents of 'scatter_gather_lab5.py', which is a Python script using MPI. The script imports MPI from mpi4py, initializes the MPI communicator, and then performs a scatter operation to distribute data to all processes. Each process then multiplies the received data by 2 and performs a gather operation to collect the results back to the root process. The final gathered result is printed.

```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5 size = comm.Get_size()
6
7 data = None
8
9 if rank == 0:
10     data = [10, 20, 30, 40]
11
12 value = comm.scatter(data, root=0)
13
14 print("Process", rank, "received", value)
15
16 value = value * 2
17
18 result = comm.gather(value, root=0)
19
20 if rank == 0:
21     print("Final gathered result:", result)
```

Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_5>mpirun -n 4 python scatter_gather_lab5.py
Process 1 received 20
Process 3 received 40
Process 2 received 30
Process 0 received 10
Final gathered result: [20, 40, 60, 80]

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_5>
```

Yes, MPI_Scatter and MPI_Gather can be used together in the same program. MPI_Scatter distributes data from the root process to all processes, while MPI_Gather collects processed data back to the root process.